Table des matières

Cours 05 — Tester, historiser et documenter 1
Objectifs 2
1. Ce qu'un test dbt est, et ce qu'il n'est pas 2
2. Les quatre tests génériques 3
3. Le test singulier, et la réconciliation qui ne voit rien 4
4. La sévérité et le seuil 4
5. Le test unitaire 5
6. Le snapshot, ou garder ce que la source ne garde pas 5
7. La dimension à évolution lente, type 2 6
8. Le fait qui pointe la version d'époque 7
9. La documentation qui ne dérive pas 8
10. Les `exposures`, ou déclarer qui consomme 8
11. La couche `PRESENTATION`, et où placer un calcul 9
Points clés à retenir 9
Exercices 10

Cours 05 — Tester, historiser et documenter

Théorie · Cours 5/9 — Formation Analytics Engineering · intermédiaire

Où vous êtes : `RAW` → `STAGING` → `INTERMEDIATE` → **`MARTS`** → **`PRESENTATION`**. Le Cours 04 a bâti l'étoile ; celui-ci prouve qu'elle dit vrai, lui rend son histoire, et la rend utilisable par d'autres que vous.

En 30 secondes :

- Un test dbt est une requête qui ne doit rien rendre. Quatre tests génériques couvrent l'essentiel, et l'un d'eux passe au vert sur un désastre.
- La sévérité et le seuil transforment un test binaire en instrument : on peut décider de savoir sans faire échouer, et de ne faire échouer qu'au-delà d'un compte connu.
- Un test unitaire vérifie la logique d'un modèle sur des entrées choisies. C'est le seul qui teste du code plutôt que des données.

© 2026 iAlexMG. Tous droits réservés.

- Un snapshot garde ce que la source ne garde pas. La dimension du concessionnaire passe en type 2, et le fait cesse de recalculer sa clé : il joint la version d'époque.
- Ce qu'on livre se documente et se déclare. `dbt docs`, le `manifest.json` et les `exposures` disent ce qui existe, d'où ça vient, et qui le consomme.

L'idée du cours. Votre étoile est juste aujourd'hui, et vous le savez parce que vous l'avez recomptée à la main. Ce cours remplace ce « vous le savez » par quelque chose qui tient sans vous : des tests qui se relancent à chaque exécution, une histoire que la source ne garde pas, et une documentation que personne n'a à tenir à jour. À la fin du labo, `dbt build` monte l'étoile et vérifie vingt-quatre choses en deux secondes, et le chiffre d'affaires par palier de 2023 aura changé, sans qu'un seul dollar ne bouge du total.

Objectifs

- Écrire un test générique, dire ce que chacun des quatre vérifie, et nommer celui qui ment.
- Régler la sévérité et le seuil d'un test, et dire à quoi cela sert.
- Écrire un test singulier, et expliquer pourquoi une réconciliation se joint en externe complète.
- Écrire un test unitaire, et dire ce qu'il attrape qu'aucun test de données n'attrape.
- Dire ce qu'un snapshot garde, comment il date ses versions, et ce que sa fréquence coûte.
- Passer une dimension en type 2, et rebrancher le fait sur la version d'époque.
- Nommer les deux pièges de la jointure temporelle, et le contrôle qui attrape chacun.
- Générer la documentation, et dire ce que le `manifest.json` contient.
- Déclarer un `exposure`, et dire à quoi il sert le jour d'une suppression.
- Placer un calcul : dans l'entrepôt, dans dbt, ou dans l'outil de restitution.

---

1. Ce qu'un test dbt est, et ce qu'il n'est pas

Un test dbt est une requête qui ne doit rendre aucune ligne. C'est tout. Si elle en rend, le test échoue, et les lignes rendues sont les coupables.

Cette définition est plus utile qu'elle n'en a l'air, parce qu'elle dit aussi ce qu'un test n'est pas. Ce n'est pas une contrainte de base de données : rien n'empêche l'écriture d'une ligne fautive, le test la constate après coup. Ce n'est pas non plus une vérification du code : un test de données regarde ce qui est dans la table, pas la logique qui l'y a mise. La section 5 s'occupe du reste.

© 2026 iAlexMG. Tous droits réservés.

dbt en distingue trois sortes, et il vaut mieux savoir laquelle employer.

La sorte	Ce que c'est	Où elle se déclare
Générique	un test paramétré, réutilisable sur n'importe quelle colonne	dans le YAML, sous la colonne
Singulier	une requête écrite pour un cas précis, non réutilisable	un fichier `.sql` dans `tests/`
Unitaire	une vérification de la logique d'un modèle sur des entrées choisies	dans le YAML, sous `unit_tests:`

Un mot de vocabulaire, parce qu'il a changé et que la documentation ancienne traîne. Le bloc du YAML s'appelle `data_tests:` depuis dbt 1.8 ; l'ancien nom, `tests:`, fonctionne encore et affiche un avertissement de dépréciation à chaque commande.

Labo : `Labo_05_AE.md`, étape 1.

2. Les quatre tests génériques

Quatre tests viennent avec dbt, et ils couvrent la grande majorité de ce qu'on veut vérifier sur une étoile.

Le test	Ce qu'il vérifie	Où on le pose chez Boréal
`unique`	aucune valeur en double	la clé de chaque dimension
`not_null`	aucune valeur absente	les clés, et toute colonne qu'un décodage produit
`accepted_values`	la colonne ne contient que les valeurs listées	le palier, la gamme
`relationships`	chaque valeur existe dans la colonne d'un autre modèle	les clés du fait vers ses dimensions

Le quatrième est celui qui rebouche le trou laissé ouvert au Cours 04. Le fait recalcule les clés de ses dimensions au lieu de les joindre, ce qui est rapide et ne garantit rien ; `relationships` est la garantie, et c'est pour cela qu'il n'est pas facultatif ici.

Le troisième, lui, ment. Il faut le savoir une fois pour toutes.

Piège — `accepted_values` passe au vert sur un décodage entièrement raté. Le test ignore les valeurs absentes, comme tous les tests génériques de dbt : il vérifie que ce qui est là fait partie de la liste, et une colonne entièrement vide ne contient rien qui n'en fasse pas partie. Mesuré sur Boréal : en cassant le décodage de la gamme, les 68 articles se retrouvent avec une gamme nulle, et `accepted_values` passe. Le `not_null` posé à côté, lui, échoue sur 68 lignes. La règle n'a pas d'exception : un `accepted_values` sans `not_null` à côté ne vérifie rien.

© 2026 iAlexMG. Tous droits réservés.

Un dernier test mérite d'être cité, et il vient de `dbt_utils` : `unique_combination_of_columns` vérifie l'unicité d'un couple de colonnes. C'est le contrôle du grain du Cours 04, écrit une fois pour toutes et relancé à chaque exécution plutôt qu'à la main.

Labo : `Labo_05_AE.md`, étapes 1, 2 et 3.

3. Le test singulier, et la réconciliation qui ne voit rien

Un test générique répond à une question générale. La réconciliation du Cours 04, elle, est propre à Boréal : la valeur nette du fait doit égaler celle des en-têtes de commande. Cela s'écrit dans un fichier `.sql` de `tests/`, et dbt attend de cette requête ce qu'il attend des autres, aucune ligne.

Reste à l'écrire correctement, et c'est là que se joue tout l'intérêt du cours.

Une réconciliation compare deux populations. La tentation est de les joindre par une jointure interne et de comparer les montants ligne à ligne. Elle attrape bien les montants qui diffèrent, et elle est aveugle à ce qui compte le plus : ce qui manque d'un côté.

Une jointure interne ne voit que ce qui existe des deux côtés. Un document absent du fait n'entre pas dans la comparaison, donc il ne produit aucune ligne, donc le test passe. Mesuré sur Boréal, en amputant le fait de janvier 2022 : la jointure interne rend zéro écart, la jointure externe complète en rend 518.

À retenir — une réconciliation se joint en externe complète, jamais autrement. La question n'est pas « les montants concordent-ils », c'est « les deux côtés portent-ils les mêmes documents, et avec les mêmes montants ». Les trois cas à sortir sont : ce qui manque à gauche, ce qui manque à droite, et ce qui diffère.

Labo : `Labo_05_AE.md`, étape 4.

4. La sévérité et le seuil

Un test qui échoue arrête la construction. C'est le comportement voulu pour une clé en double ; ce ne l'est pas toujours.

Trois réglages changent cela, et ils se posent sous `config:` du test.

Le réglage	Ce qu'il fait
`severity`	`error`, le défaut, ou `warn`, qui signale sans arrêter
`warn_if`	la condition qui déclenche l'avertissement, par exemple `">23"`
`error_if`	la condition qui déclenche l'échec

Ce que cela permet est plus intéressant qu'un simple assouplissement. Chez Boréal, 23 articles n'ont pas d'année-modèle, et c'est normal : ce sont les pièces et accessoires. Un `not_null` échouerait tous les jours pour rien, et on le retirerait, ce qui reviendrait à ne plus rien surveiller.

Avec un seuil, la colonne reste surveillée sans faire de bruit : `warn_if: ">23"` laisse passer les 23 connus et signale le 24e. Mesuré : à 23 échecs le test passe en silence ; en abaissant le seuil à 22, dbt rend `WARN 23 ... configured to warn if >22`.

© 2026 iAlexMG. Tous droits réservés.

Pourquoi — un seuil documente un fait, un test retiré ne documente rien. La valeur du seuil est une affirmation sur les données, qu'on écrit dans le projet et qui se relit : « il y en a 23, et nous voulons le savoir si ça change ». Retirer le test aurait la même tranquillité et aucune de l'information.

Labo : `Labo_05_AE.md`, étape 5.

5. Le test unitaire

Les tests de données regardent le contenu d'une table. Ils ne disent rien de la logique qui l'a produite, et il y a des logiques qu'aucune donnée ne suffit à éprouver.

Le cas type est une condition aux bornes. La jointure temporelle de la section 8 doit choisir la bonne version d'un concessionnaire selon la date d'une vente ; un test de données constatera que chaque ligne a bien une clé, jamais qu'une vente tombée exactement le jour du changement a reçu la bonne des deux versions. Il faudrait pour cela qu'une telle vente existe dans les données, ce qui n'est ni garanti ni souhaitable comme méthode.

Un test unitaire, arrivé avec dbt 1.8, retourne la question. On donne au modèle des entrées écrites à la main, on déclare la sortie attendue, et dbt compare.

La partie	**Ce qu'elle porte**
`model`	le modèle testé
`given`	une liste d'entrées, une par `ref()` ou `source()` que le modèle lit
`expect`	les lignes attendues en sortie

Deux précisions valent la peine d'être connues avant d'en écrire un. Les colonnes qu'on ne cite pas dans `expect` ne sont pas comparées, ce qui permet de tester une logique sans recopier tout le modèle. Et quand le test échoue, dbt rend un tableau qui montre la ligne fautive et la colonne, sous la forme `attendu→obtenu`, ce qui en fait un bon outil d'enseignement autant que de contrôle.

Piège — un test unitaire ne peut pas lire les colonnes d'un modèle `ephemeral`. Pour préparer les entrées, dbt va chercher les types des colonnes dans la base ; un modèle `ephemeral` n'y a aucun objet, et le test s'arrête sur « Not able to get columns for unit test … because the relation doesn't exist ». La parade tient en une ligne : donner cette entrée en `format: sql`, c'est-à-dire par un `select` qui déclare ses propres types, ce qui dispense dbt d'aller les chercher.

Labo : `Labo_05_AE.md`, étape 9.

---

6. Le snapshot, ou garder ce que la source ne garde pas

Ici commence la seconde séance de lecture.

© 2026 iAlexMG. Tous droits réservés.

La plupart des systèmes sources écrasent. Le concessionnaire passe de Bronze à Or, et la fiche dit Or ; ce qu'elle disait la veille n'existe plus nulle part. Si l'entrepôt se contente de lire la source, il hérite de cet oubli.

Un snapshot est la parade. C'est une table que dbt tient lui-même : à chaque exécution de `dbt snapshot`, il compare ce que la source dit maintenant à ce qu'il avait enregistré, ferme les lignes qui ont changé, et en ouvre de nouvelles. Quatre colonnes apparaissent.

La colonne	Ce qu'elle porte
`dbt_scd_id`	l'identifiant de la version
`dbt_updated_at`	la valeur qui a servi à dater le changement
`dbt_valid_from`	le début de validité de cette version
`dbt_valid_to`	la fin, ou une absence pour la version courante

Deux stratégies décident de ce qui compte comme un changement. La stratégie `timestamp` fait confiance à une colonne de date de modification de la source. La stratégie `check` compare les valeurs d'une liste de colonnes, et c'est celle qu'on emploie quand la source n'a pas de date de modification fiable, ce qui est le cas courant.

Reste une question que la documentation traite vite et qui décide de tout : à quelle date dbt ouvre une version. Par défaut, à l'heure du lancement, ce qui veut dire que votre histoire est datée des jours où vous avez regardé. Pour Boréal ce serait inutilisable, puisqu'on veut rejouer deux ans d'histoire aujourd'hui. Une configuration `updated_at` résout le cas : elle désigne une colonne de la requête, et c'est elle qui date les versions. Mesuré : avec `updated_at` sur la date de l'extrait mensuel, `dbt_valid_from` et `dbt_valid_to` portent des dates d'affaires, du type `DATE`, et non l'heure du lancement.

À retenir — un snapshot est une table, pas une vue. Il ne change pas quand sa source change, il change quand on le lance. Mesuré sur Boréal : en cassant le décodage du palier dans le staging, la dimension et ses tests restent entièrement au vert, parce qu'ils lisent le snapshot, qui porte encore les anciennes valeurs. C'est une protection et un piège : la donnée fausse entrera au prochain `dbt snapshot`, pas avant.

Et une conséquence que le labo vous fera constater : la précision d'un snapshot est celle de sa fréquence. Chez Boréal, les paliers ont été révisés le 1er juillet 2024, mais l'extrait qui le montre est celui de juillet, daté du 31. Les ventes du mois de juillet sont donc attribuées à l'ancien palier, pour 4 314 525,62 $. Ce n'est pas une erreur du snapshot, c'est ce qu'il sait : il a vu le changement le 31, parce qu'il n'a regardé que ce jour-là.

Labo : `Labo_05_AE.md`, étape 6.

7. La dimension à évolution lente, type 2

Le Cours 02 a posé le choix, le Cours 04 a livré une dimension de type 1, et son `LIVRAISON.md` a écrit ce que cela coûtait. Voici la moitié qui manquait.

Une dimension de type 2 porte une ligne par version : le concessionnaire `0000200009` y figure deux fois, Bronze jusqu'au 31 juillet 2024, Argent depuis. Chez Boréal, les 156 concessionnaires donnent 205 lignes, dont 49 fermées, et ces 49 sont exactement la révision des paliers.

© 2026 iAlexMG. Tous droits réservés.

Trois choses changent dans le modèle, et la deuxième est celle qu'on oublie.

La clé de substitution ne se hache plus sur le seul code : elle se hache sur le code et la date d'ouverture de la version. Sans quoi deux versions du même concessionnaire auraient la même clé, et la dimension ne serait pas unique.

Les colonnes qu'on ne suit pas restent en type 1, et il faut le dire. Le palier est historisé ; le nom, la ville et la province viennent de l'état courant du staging, donc une version fermée affiche le nom d'aujourd'hui. C'est un choix défendable, et il cesse de l'être le jour où personne ne sait qu'il a été fait.

Enfin, la première version de chaque entité s'ouvre à une date plancher, très ancienne, et non à la date où le snapshot l'a vue pour la première fois. La raison est arithmétique : une vente antérieure au premier regard ne trouverait aucune version, et disparaîtrait du fait. Chez Boréal, sans ce plancher, 1 894 lignes sortent de l'étoile, dont 1 418 en janvier 2022.

Labo : `Labo_05_AE.md`, étape 7.

8. Le fait qui pointe la version d'époque

C'est ici que le Cours 04 se retourne. Le fait y recalculait la clé du concessionnaire en hachant son code ; il ne le peut plus, puisque le code ne suffit plus à désigner une ligne de la dimension. Il faut joindre, et joindre sur une condition de date.

```
  inner join dim_concessionnaire as d
      on d.code_concessionnaire = l.code_concessionnaire
     and l.date_commande >= d.valide_de
     and (d.valide_jusqua is null or l.date_commande < d.valide_jusqua)
```

Deux pièges guettent cette jointure, et le cours vous les fera rencontrer tous les deux, parce que les contrôles du Cours 04 les attrapent chacun.

Le premier est l'intervalle. dbt ferme une version à la date où la suivante s'ouvre : le 31 juillet 2024 est à la fois la fin de l'ancienne et le début de la neuve. Une condition inclusive des deux côtés fait donc correspondre deux versions à une vente de ce jour-là, et le fait se dédouble. Mesuré : 56 lignes de commande portent cette date, 16 se dédoublent, et le contrôle du grain tombe immédiatement, 73 774 lignes pour 73 758 couples. La parade est un intervalle semi-ouvert, fermé au début et ouvert à la fin.

Le second est le plancher de la section précédente, et c'est la réconciliation qui l'attrape. Avec l'intervalle corrigé mais sans le plancher, le grain redevient juste et la valeur nette manque de 9 455 090,05 $. Un chiffre d'affaires de 420 millions au lieu de 430 est parfaitement plausible, et seul le témoin de l'ERP le contredit.

À retenir — les deux contrôles du Cours 04 attrapent chacun un défaut, et aucun n'attrape les deux. Le contrôle du grain voit le dédoublement et ne voit pas les lignes perdues ; la réconciliation voit les lignes perdues et ne voit pas le dédoublement, puisqu'une ligne en double ajoute de la valeur là où l'autre défaut en retire. C'est la raison pour laquelle on en pose deux.

© 2026 iAlexMG. Tous droits réservés.

Quand les deux sont corrigés, le fait retrouve ses 75 652 lignes et ses 430 148 012,16 $, au cent près. Et c'est le résultat le plus important du cours : l'historisation change la façon dont la valeur est attribuée, jamais son total. Chez Boréal, 17 733 lignes changent de palier selon qu'on lit la version d'époque ou celle d'aujourd'hui, pour 94 628 792,88 $, et le total ne bouge pas d'un cent.

Labo : `Labo_05_AE.md`, étape 8.

9. La documentation qui ne dérive pas

Une documentation écrite à la main est fausse trois mois plus tard. Celle de dbt est extraite du projet lui-même, donc elle ne peut dire que ce qui existe.

`dbt docs generate` produit deux fichiers, et un site pour les lire.

Le fichier	Ce qu'il porte
`manifest.json`	tout ce que dbt sait du projet : les modèles, leur SQL compilé, le graphe, les tests, les descriptions
`catalog.json`	ce que la base sait des objets produits : les colonnes et leurs types
`index.html`	le site, qui lit les deux

Le `manifest.json` mérite plus qu'une ligne, parce qu'il ressert deux fois dans le parcours. C'est le seul endroit où le lignage existe sous une forme qu'un programme peut lire : le Cours 08 s'en sert pour qu'un agent d'intelligence artificielle sache ce que l'entrepôt contient, et la formation Architecture et Gouvernance y lit de quoi remonter d'un rapport jusqu'à la source. Sur le projet de Boréal, il pèse environ 870 Ko pour douze modèles.

Les descriptions que vous écrivez dans le YAML y entrent, et c'est le seul travail de documentation qui reste à faire à la main. Une description utile dit ce que la table est et ce qu'elle n'est pas ; une description qui répète le nom du modèle ne vaut pas la ligne qu'elle occupe.

Labo : `Labo_05_AE.md`, étape 10.

10. Les `exposures`, ou déclarer qui consomme

Le graphe de dbt s'arrête à la sortie de votre projet. Ce que quelqu'un en fait ensuite, un tableau de bord, un modèle, un export, dbt l'ignore, et la question qui compte le jour où vous voulez supprimer une colonne reste sans réponse.

Un `exposure` est la déclaration de ce qui vit en aval. On le déclare en YAML, avec son type, ses dépendances et son propriétaire, et il entre dans le graphe comme un nœud terminal.

Trois choses en découlent. `dbt ls --select +<exposure>` dit tout ce dont un tableau de bord dépend. La documentation le montre, donc un analyste voit à quoi sert la table qu'il regarde. Et une sélection comme `dbt build --select +tableau_de_bord_des_ventes` reconstruit exactement ce qu'il faut avant une publication.

© 2026 iAlexMG. Tous droits réservés.

Piège — le propriétaire d'un `exposure` est une équipe, jamais une personne nommée. Un nom personnel périme au premier départ, et il transforme un document de projet en annuaire. On écrit le nom de l'équipe et une adresse de courriel de groupe.

Labo : `Labo_05_AE.md`, étape 11.

11. La couche `PRESENTATION`, et où placer un calcul

L'étoile est faite pour être interrogée, pas pour être lue telle quelle. Entre elle et un rapport, il reste souvent un dernier modèle : un résumé au grain du rapport, avec les jointures déjà faites et les mesures déjà additionnées. C'est la couche `PRESENTATION`, et ses modèles portent le préfixe `rpt_`.

Ce qu'un modèle `rpt_*` a le droit de faire tient en trois lignes. Il joint le fait à ses dimensions. Il agrège à un grain plus grossier. Il ne crée aucune mesure que l'étoile ne porte pas déjà, et il ne corrige rien : un chiffre faux se corrige dans le modèle qui le produit.

Reste l'arbitrage qui revient dans tous les projets : où placer un calcul, entre l'entrepôt, dbt et l'outil de restitution.

Le calcul	Où il va	Pourquoi
Une règle de métier, un filtre, une jointure	dans dbt	elle doit valoir pour tout le monde, et se tester
Une agrégation qu'un rapport relit souvent	dans un `rpt_*`	elle se calcule une fois plutôt qu'à chaque ouverture
Un ratio, un pourcentage, un écart à une cible	dans l'outil de restitution	il dépend du niveau d'agrégation affiché

La troisième ligne est celle que le Cours 04 a démontrée par l'absurde : un taux stocké se moyenne, et une moyenne de taux ne veut rien dire. Le résumé du labo porte donc la marge et la valeur nette, jamais leur rapport ; le rapport divise à la lecture, au niveau qu'il affiche.

Le reste de la restitution appartient à Business Intelligence : le modèle sémantique, les mesures nommées, la mise en forme. La frontière est nette et elle se dit en une phrase : ici on livre des tables et des colonnes, là-bas on livre des définitions et des écrans.

Recoupements. Les contrats d'entrée du Cours 11 de Data Engineering et les tests dbt sont le même geste à deux endroits de la chaîne : l'un vérifie ce qui arrive, l'autre ce qu'on produit. La formation Architecture et Gouvernance, à son Cours 05, fait du contrat un engagement entre équipes, avec un propriétaire et un recours, ce qu'un test seul n'est pas. Et la question de savoir où vit la définition d'un indicateur appartient au Cours 06 de Business Intelligence : ici on s'arrête au modèle `rpt_*` et à l'arbitrage du lieu de calcul.

Labo : `Labo_05_AE.md`, étape 12.

Points clés à retenir

- Un test dbt est une requête qui ne doit rien rendre. Les tests de données regardent le contenu, les tests unitaires regardent la logique.

© 2026 iAlexMG. Tous droits réservés.

- `accepted_values` passe au vert sur une colonne entièrement vide : il se pose toujours avec un `not_null` à côté.
- `relationships` est la garantie que le Cours 04 avait laissée manquante en recalculant les clés.
- Une réconciliation se joint en externe complète : une jointure interne ne voit pas ce qui manque.
- Un seuil documente un fait connu ; un test retiré ne documente rien.
- Un test unitaire éprouve les bornes d'une logique, que les données n'éprouvent que par hasard.
- Un snapshot garde ce que la source écrase. C'est une table : il change quand on le lance, pas quand la source change, et sa précision est celle de sa fréquence.
- Une dimension de type 2 porte une ligne par version, sa clé se hache sur le code et la date d'ouverture, et sa première version s'ouvre à un plancher.
- La jointure temporelle se fait sur un intervalle semi-ouvert. Le contrôle du grain attrape le dédoublement, la réconciliation attrape les lignes perdues, et aucun n'attrape les deux.
- L'historisation change l'attribution de la valeur, jamais son total.
- La documentation sort du projet, et le `manifest.json` est le seul lignage qu'un programme peut lire.
- Un `exposure` déclare qui consomme, et son propriétaire est une équipe.
- Un ratio ne se stocke pas : il se calcule au niveau où il s'affiche.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_05_AE.md`. Faites-les après le labo : plusieurs énoncés portent sur des chiffres que vous aurez vus bouger.

© 2026 iAlexMG. Tous droits réservés.